

中山大學

SUN YAT-SEN UNIVERSITY

实验 0x07 实验报告

更好的内存管理

姓　　名：　何亮

学　　号：　24325005

教学班号：　202520804（陈鹏飞老师班级）

专　　业：　计算机科学与技术

院　　系：　计算机学院

2025 学年第二学期

1. 帧分配器的内存回收

我们在 lab-4 的加分项中已经实现了帧分配器的内存回收，即用一个 `Vec<PhysFrame>` 作为回收的内存池，保存每次释放了的帧，然后每次 allocate 一个新的帧的时候先检查内存池种是否有可用的帧，如下

```
unsafe impl FrameAllocator<Size4KiB> for BootInfoFrameAllocator {
    fn allocate_frame(&mut self) -> Option<PhysFrame> {
        // FIXED: try pop from recycled frames
        self.used += 1;
        if let Some(reused_frame) = self.recycled.pop() {
            return Some(reused_frame);
        }
        // FIXED: if recycled is empty:
        //        try allocate from frames like before
        self.frames.next()
    }
}

impl FrameDeallocator<Size4KiB> for BootInfoFrameAllocator {
    unsafe fn deallocate_frame(&mut self, _frame: PhysFrame) {
        // FIXED: push frame to recycled
        self.recycled.push(_frame);
        self.used -= 1;
    }
}
```

2. 用户程序的内存统计

我们在本实验中只考虑堆内存，栈内存，以及 ELF 文件映射占用的内存，而不考虑页表等占用的内存。而前面两者我们已经实现了，因此我们这部分只需要考虑统计 ELF 文件映射占用的内存。

我们考虑在 `load_elf` 加载 ELF 文件的时候就进行记录，这里根据文档将 `load_elf` 修改为用一个 `Vec<PageRangeInclusive>` 记录所有加载的 ELF 文件映射的页表范围。

随后，我们在 `load_elf_code` 函数中，将返回的 `Vec<PageRangeInclusive>` 进行计算大小并求和，得到该部分占用的内存

```
fn load_elf_code(&mut self, elf: &ElfFile, mapper: MapperRef, alloc:
FrameAllocatorRef) {
    // FIXED: make the `load_elf` function return the code pages
    self.code =
        elf::load_elf(elf, *PHYSICAL_OFFSET.get().unwrap(), mapper, alloc,
true).unwrap();

    // FIXED: calculate code usage
    self.code_usage = self
        .code
        .iter()
        .map(|range| (range.count() as u64) * crate::memory::PAGE_SIZE)
        .sum();
}
```

随后, 我们修改 `print_process_list`, 加入对内存信息的统计, 由于我们实际使用的内存记录在 `used` 成员里面, 而且在每次 deallocate 的时候减去了, 因此我们这里计算只需要算 `used` 成员即可, 如下

```
// FIXED: print memory usage of kernel heap
let alloc = get_frame_alloc_for_sure();
let frames_used = alloc.frames_used();
let frames_total = alloc.frames_total();

let used = frames_used * PAGE_SIZE as usize;
let total = frames_total * PAGE_SIZE as usize;

output += &format_usage("Memory", used, total);
drop(alloc);
```

我们可以看到, 目前连续运行两次 factorial 的结果如下,

```
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |    4769 |  32.0 KiB | Running
 #  1 | #  0 | kernel       |    4827 |   2.0 MiB | Ready
Memory :   1.08 MiB /  44.50 MiB ( 2.43%)
Queue  : [1]
CPUs   : [0: 2]
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |  695366 |  32.0 KiB | Blocked
 #  3 | #  2 | /app/factor~1 |    7088 |  15.3 MiB | Running
 #  1 | #  0 | kernel       |  702511 |   2.0 MiB | Ready
Memory :  16.43 MiB /  44.50 MiB (36.93%)
Queue  : [1]
CPUs   : [0: 3]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [3] exited with code: 0
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |  730795 |  32.0 KiB | Blocked
 #  1 | #  0 | kernel       |  746605 |   2.0 MiB | Ready
 #  4 | #  2 | /app/factor~1 |    8658 |  15.3 MiB | Running
Memory :  31.79 MiB /  44.50 MiB (71.43%)
Queue  : [1]
CPUs   : [0: 4]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [4] exited with code: 0
```

这是由于我们还没有实现进程退出时内存的释放, 因此此时占用的大小是叠加的, 运行两次之后变成了两倍。

3. 用户程序的内存释放

首先，我们需要在 `stack.rs` 中实现栈的内存释放函数 `clean_up`，代码如下

```rust
pub fn clean_up(
    &mut self,
    mapper: MapperRef,
    dealloc: &mut impl x86_64::structures::paging::FrameDeallocator<Size4KiB>,
) -> Result<(), x86_64::structures::paging::mapper::UnmapError> {
    let count = self.usage;
    if count == 0 {
        warn!("Stack is empty, no need to clean up.");
        return Ok(());
    }

    // The stack range might have more pages mapped if grown,
    // we unmap the inclusive range.
    let range_inclusive = Page::range_inclusive(self.range.start, self.range.end
- 1);
    elf::unmap_range(range_inclusive, mapper, dealloc, true)?;
    self.range = Page::range(self.range.start, self.range.start);

    self.usage = 0;

    Ok(())
}
```

由于我们的内存范围定义是左闭右开的，因此我们要先转化为闭区间 `range_inclusive` 然后调用 `elf::unmap_range()` 进行释放，随后还需要更新栈占用的页面为 0 以及内存的页面范围为一个空区间。

然后我们再在 `proc/vm/mod.rs` 的 `clean_up` 函数中，根据文档加入相关调试信息的输出。

最后，我们需要对 `ProcessVm` 实现 `Drop` trait，调用自己的 `.clean_up()` 函数释放内存，此时 `kill` 函数中调用的 `proc_vm.take()` 在进程退出的时候释放了 `ProcessVm` 的所有权，当 `ProcessVm` 对象没有被任何一个对象持有时，就会自动调用我们实现的 `Drop` trait 中的 `drop` 函数，从而实现了进程退出时的内存释放。

我们可以运行 `factorial`，发现在进程退出之后，使用的内存也被正常释放了！

```
[>] factor~1
Input n: 999999
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  1 | #  0 | kernel       |  119398 |   2.0 MiB | Ready
 #  2 | #  1 | /app/sh      |  112529 |  32.0 KiB | Blocked
 #  3 | #  2 | /app/factor~1 |    6841 |  15.3 MiB | Running
Memory :  16.43 MiB /  44.49 MiB (36.94%)
Queue  : [1]
CPUs   : [0: 3]
The factorial of 999999 under modulo 1000000007 is 128233642.
Process [3] exited with code: 0
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  1 | #  0 | kernel       |  126421 |   2.0 MiB | Ready
 #  2 | #  1 | /app/sh      |  119547 |  32.0 KiB | Running
Memory :   1.10 MiB /  44.49 MiB ( 2.47%)
Queue  : [1]
CPUs   : [0: 2]
```

4. 内核的内存统计

内核的内存占用包括了内核栈内存的占用以及加载 ELF 文件使用的内存，前者我们已经统计，因此这里只需要实现后者的统计。

我们的方法仍然是在加载内核的 ELF 时就记录下来占用的内存，并记录在 `bootinfo` 中，在初始化内核的时候，把这部分的占用记录到内核的 `ProcessVm` 对象中的 `.code` 和 `.code_usage` 成员中即可。

我们可以观察到，相比于之前的内核占用为 `2.0MiB`，现在变为了 `2.3MiB`，这和 ELF 中 Load 类型的非 .bss 段占用的总内存，即 `0x4b00` Bytes，约为 `0.293MiB` 对应，说明我们成功实现了内核的 ELF 文件的统计！

```
            FileSiz            MemSiz              Flags  Align
LOAD           0x0000000000001000 0xffffff0000000000 0xffffff0000000000
                0x000000000000e084 0x000000000000e084  R      0x1000
LOAD           0x0000000000010000 0xffffff000000f000 0xffffff000000f000
                0x0000000000036bd7 0x0000000000036bd7  R E    0x1000
LOAD           0x0000000000047000 0xffffff0000046000 0xffffff0000046000
                0x0000000000004148 0x0000000000004148  RW     0x1000
```

```
Welcome to YatSenOS Shell!
Type 'help' for instructions.
[>] ps
  PID | PPID | Process Name |  Ticks  |   Memory  | Status
 #  2 | #  1 | /app/sh      |   11394 |  32.0 KiB | Running
 #  1 | #  0 | kernel       |   11424 |   2.3 MiB | Ready
Memory :   1.08 MiB /  44.49 MiB ( 2.43%)
Queue  : [1]
CPUs   : [0: 2]
```

5. 内核栈的自动增长